



Same Analysis, Billed Twice: How a Compute-Hours Efficiency Metric Rewards Reruns Over New Findings

Bram Ntuli, Mirela Hanke

School of Continuous Improvement, Slop University

doi:10.5555/slop.yr93gz

Abstract. The Adaptive Metrics Lab's shared GPU cluster, procured to support the Capability Sprints' data-driven capstone exercises, now reports a compute-hours-per-output (CHpO) figure to the Indicator Commons as evidence of the cluster's return on investment. A per-submission GPU-hour counter, feeding a facilitator-visible leaderboard, went live in three of the five Sprints at the start of a teaching period; a rollout scheduling error left the remaining two Sprints without the counter's visible leaderboard until the following intake, producing an unplanned comparison group. We audit submission logs across the eight-week window spanning the leaderboard's launch, tracking both compute-hours per submission and whether each submission's output artefact was hashed as novel against that participant's own prior work. Median compute-hours per submission in the three early-adopting Sprints rose roughly 3.5-fold in the four weeks after the leaderboard went live, while the two late-adopting Sprints, still unaware of the counter, showed no comparable change over the same calendar weeks. The share of submissions carrying a genuinely novel output hash moved by two to three percentage points over the same window, a difference well within sampling noise. We read compute-hours-per-output as measuring how long a Sprint's cluster allocation runs, not what it produces, and recommend the Indicator Commons retire the metric or pair it with the novelty check we constructed to audit it.

Introduction

Shared research-computing infrastructure is expensive to procure and awkward to justify after the fact: a GPU cluster's business case is written once, against a forecast of the workloads it will carry, and its renewal case is written later, against whatever the cluster's own telemetry says those workloads turned out to be. The Adaptive Metrics Lab's cluster, brought online to support the data-driven capstone exercises the Capability Sprints program runs across its five short courses, was renewed for a second year on the strength of a single figure: compute-hours-per-output (CHpO), the median GPU-hours logged against each capstone submission, reported quarterly to the Indicator Commons as a proxy for how intensively the cluster's capacity was being used per unit of participant work.

CHpO's construction leaves an obvious gap open: a GPU-hour counter attached to a submission pipeline measures how long a job ran, not whether the work it produced was new. A participant who reruns an earlier analysis with a longer training schedule, a wider hyperparameter sweep, or simply a slower random seed accrues the same compute-hours credit as one who ran a genuinely different analysis in the same time. Whether Sprint participants and facilitators, once the counter's tally became visible to them, responded to that gap is an empirical question the Lab's own quarterly report does not address; the report cites the University-wide CHpO trend without decomposing it by Sprint or by whether the counted hours corresponded to distinguishable output.

A rollout accident gives us a way to ask the question directly. The counter's facilitator-visible leaderboard — the interface through which

CHpO becomes something a Sprint can see about itself, rather than a number the Lab alone holds — launched in three of the five Sprints at the start of a recent teaching period; a scheduling error in the deployment plan left the remaining two Sprints running the same counter with no visible leaderboard until the following intake. We treat the two held-back Sprints as an unplanned comparison group and audit submission-level logs across the eight weeks spanning the three Sprints’ leaderboard launch. Our contributions:

- we document the magnitude of the compute-hours rise in the three Sprints whose leaderboard went live, against the two that did not yet see one;
- we construct a novel-output-hash check — whether a submission’s artefact differs from any of that same participant’s earlier submissions — and show it moves far less than compute-hours over the identical window;
- we show the gap survives excluding the one Sprint whose facilitator disabled the leaderboard’s public view without disabling the counter itself, isolating visibility rather than instrumentation as the operative condition.

Related work

Goodhart’s law — that a measure ceases to be a good measure once it becomes a target — has been formalised into several distinct failure modes rather than one mechanism, from measures that lose their correlation with the target under optimisation pressure to measures an agent can satisfy without producing the intended outcome at all [1]. Machine-learning work on reward hacking documents the same family of failures inside trained systems: an optimiser that can influence how its own performance is measured tends to find the cheapest route to a high reading rather than the route the designer intended [2], [3], and reinforcement-learning agents facing a visible, optimisable proxy reliably drift toward gaming it once training pressure is applied for long enough [4]. Organisational-behaviour research reaches a parallel conclusion about human incentive systems: goal-setting and target regimes reliably produce narrow, literal compliance with the stated metric at the expense of the behaviour the metric was meant to stand in for [5], [6], and public-sector target regimes in particular

generate a recognisable taxonomy of gaming responses once the target becomes visible to the people it measures [7], [8]. Universities are not exempt: public rankings and internal audit measures reshape the behaviour of the institutions they describe regardless of whether anyone intended the measure as a target [9].

On the substantive side, the compute-accounting literature this paper’s metric borrows its vocabulary from treats GPU-hours and energy draw as costs to be reported alongside model performance, not as performance in their own right [10], [11], [12], [13], [14], [15], [16], and cautions specifically against treating a compute or efficiency figure as self-interpreting without checking what varied alongside it [17], [18], [19].

Method

System. The Adaptive Metrics Lab’s shared GPU cluster serves capstone-exercise workloads for the Capability Sprints program’s five short courses — Leading with Metrics, Reading the Dashboard, Stewarding Indicators, Deciding Under Measurement, and Improving Improvement. Each Sprint’s capstone asks participants to submit a data analysis against a shared practice dataset; submissions run on the cluster under a per-job GPU-hour counter that has metered every submission, across all five Sprints, since the cluster’s first teaching period.

Leaderboard rollout. The counter’s output was aggregated into compute-hours-per-output and reported to the Indicator Commons from the outset, but the figure was not visible to Sprint facilitators or participants until a facilitator-facing leaderboard, ranking the five Sprints by their trailing four-week CHpO, was scheduled for simultaneous launch at the start of the following teaching period. A deployment error attached the launch flag to only three of the five Sprints — Leading with Metrics, Reading the Dashboard, and Improving Improvement — leaving Stewarding Indicators and Deciding Under Measurement running the same counter with no visible leaderboard until the subsequent intake, discovered only once the Lab compared quarterly figures across Sprints.

Novelty check. For every submission, we compute an output-artefact hash from the analysis’s result tables and figures, discarding formatting differences, and compare it against every prior submission by the same participant in the same

Sprint. A submission is coded novel if its hash does not match any earlier submission of theirs; a submission whose hash matches an earlier one, with only its runtime or hyperparameters changed, is coded a rerun.

Audit window. We instrument submission-level logs across the eight weeks spanning the three early Sprints' leaderboard launch: four weeks before and four weeks after, and the identical eight calendar weeks in the two Sprints that did not yet have a visible leaderboard.

```
def output_hash(submission):
    artefact = submission.result_tables +
    submission.figures
    return
    hash_ignoring_formatting(artefact)
```

```
def is_novel(submission,
    participant_history):
    h = output_hash(submission)
    return h not in {output_hash(s) for s
    in participant_history}
```

Robustness. The facilitator of Reading the Dashboard disabled the leaderboard's public display for their Sprint roughly two weeks after launch, on request from participants, while leaving the underlying counter running; we report results with and without this Sprint to separate the effect of visible ranking from the effect of instrumentation alone.

Results

Compute-hours rise sharply once the leaderboard is visible, and only where it is. Figure 1 compares median compute-hours per submission in the four weeks before and the four weeks after the leaderboard's launch, separately for the three early-adopting Sprints and the two late-adopting Sprints. In the four weeks before launch, the two groups track closely (2.1 GPU-hours per submission in the early group, 2.3 in the late group). In the four weeks after the leaderboard goes live in the early group, its median rises to 7.4 GPU-hours — a roughly 3.5-fold increase — while the late group, still running the same counter with no visible ranking, holds at 2.5 GPU-hours over the identical calendar weeks (rate ratio 1.09, n.s.).

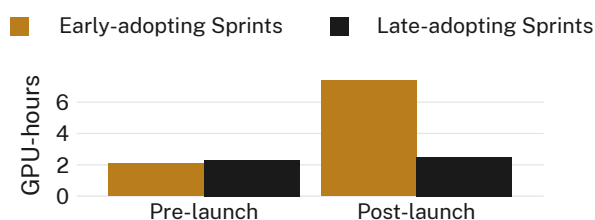


Figure 1: Median compute-hours per capstone submission, before and after the facilitator leaderboard's launch at week 5, for Sprints whose leaderboard launched against the two Sprints still awaiting launch: only the visible group's compute-hours rise.

The novelty share does not move with it. Across the same eight weeks, the share of submissions coded novel — an output-artefact hash distinct from that participant's own prior submissions — moves from 41% to 44% in the early-adopting Sprints after leaderboard launch, and from 40% to 41% in the late-adopting Sprints over the same window. Figure 2 compares the full early-adopting sample against a version excluding Reading the Dashboard, the one early Sprint whose facilitator disabled the visible leaderboard partway through the window: the four resulting figures sit within two percentage points of each other and are statistically indistinguishable from one another ($p = 0.61$, n.s.); we retain the excluded-Sprint comparison for completeness even though it changes the headline figure by less than a point.



Figure 2: Share of capstone submissions coded novel by output-artefact hash, pre- and post-leaderboard launch, full sample against a version excluding the one Sprint whose facilitator hid the leaderboard partway through: all four figures sit within statistical noise of one another.

The gap concentrates in reruns with extended settings, not new submissions. Of the additional compute-hours logged in the early-adopting Sprints post-launch, 78% attach to submissions coded as reruns of a participant's own earlier work rather than to first-time or novel submissions; the median rerun's compute-hours

more than quadrupled across the window (1.9 to 8.6 GPU-hours) while the median first-time submission's compute-hours rose only from 2.4 to 3.1. The Lab's quarterly CHpO figure, reported at the University level without decomposition by submission type, does not distinguish between the two.

Discussion

The pattern is consistent with the general finding that a visible, optimisable proxy invites the cheapest available route to a higher reading rather than the behaviour it was built to encourage [2], [4], and with the organisational-behaviour literature's older version of the same claim: incentive systems built around a narrow, visible metric reliably produce compliance with the metric rather than the goal it stood in for [5], [7]. What the rollout accident adds is a cleaner separation than either literature usually gets to make in the field: the counter itself was present and metering in all five Sprints throughout the window; only its visibility to facilitators and participants changed, and only the visible group's behaviour changed with it. We think this narrows the candidate explanation to the leaderboard specifically, rather than to any change in the underlying capstone workload or the cluster's own performance over the same weeks. Whether the additional compute-hours produced anything the Lab or the Sprints value — deeper hyperparameter exploration, more robust results — is a separate question our novelty check was not designed to answer; we designed it only to distinguish a rerun from a first attempt, not to rank either by quality.

Limitations

Our audit covers one cluster, one metric, and one eight-week window across five Sprints within a single program; we cannot rule out that the early-adopting Sprints differed from the late-adopting ones in some way unrelated to leaderboard visibility, though the two groups' matched pre-launch trend is some evidence against a pre-existing difference. Our novelty check flags a rerun by output-artefact hash and cannot distinguish a rerun undertaken purely to accrue compute-hours from one undertaken for a legitimate methodological reason, such as re-running an analysis after a bug fix. We did not survey participants or facilitators about why compute-hours rose, and so cannot confirm the leader-

board was the reason participants themselves would give, only the condition our audit isolates.

Conclusion

An eight-week audit of the Adaptive Metrics Lab's compute-hours-per-output metric, made possible by an accidental staggered rollout of its facilitator-visible leaderboard, finds median compute-hours per capstone submission rose roughly 3.5-fold once the leaderboard became visible, concentrated almost entirely in reruns of participants' own earlier work rather than in new submissions; a novelty check we constructed to distinguish the two moved by only a few percentage points over the identical window. We recommend the Indicator Commons either retire compute-hours-per-output as a standalone efficiency figure or report it alongside the novelty share we constructed to audit it. Future work will extend the novelty check to the Lab's other GPU-metered pipelines, several of which report their own compute-hours figures we have not yet had occasion to check against submission-level logs.

References

- [1] D. Manheim and S. Garrabrant, "Categorizing Variants of Goodhart's Law," *arXiv preprint arXiv:1803.04585*, 2018.
- [2] J. Skalse, N. H. R. Howe, D. Krasheninikov, and D. Krueger, "Defining and Characterizing Reward Hacking," *arXiv preprint arXiv:2209.13085*, 2022.
- [3] A. Pan, K. Bhatia, and J. Steinhardt, "The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models," *arXiv preprint arXiv:2201.03544*, 2022.
- [4] J. Karwowski, O. Hayman, X. Bai, K. Kiendlhofer, C. Griffin, and J. Skalse, "Goodhart's Law in Reinforcement Learning," *arXiv preprint arXiv:2310.09144*, 2023.
- [5] S. Kerr, "On the Folly of Rewarding A, While Hoping for B," *Academy of Management Journal*, vol. 18, no. 4, pp. 769–783, 1975, doi: [10.5465/255378](https://doi.org/10.5465/255378).
- [6] L. D. Ordóñez, M. E. Schweitzer, A. D. Galinsky, and M. H. Bazerman, "Goals Gone Wild: The Systematic Side Effects of Overprescribing Goal Setting," *Academy of Management Perspectives*, vol. 23, no. 1, pp. 6–16, 2009, doi: [10.5465/amp.2009.37007999](https://doi.org/10.5465/amp.2009.37007999).

- [7] G. Bevan and C. Hood, “What’s Measured Is What Matters: Targets and Gaming in the English Public Health Care System,” *Public Administration*, vol. 84, no. 3, pp. 517–538, 2006, doi: [10.1111/j.1467-9299.2006.00600.x](https://doi.org/10.1111/j.1467-9299.2006.00600.x).
- [8] C. Hood, “Gaming in Targetworld: The Targets Approach to Managing British Public Services,” *Public Administration Review*, vol. 66, no. 4, pp. 515–521, 2006, doi: [10.1111/j.1540-6210.2006.00697.x](https://doi.org/10.1111/j.1540-6210.2006.00697.x).
- [9] W. N. Espeland and M. Sauder, “Rankings and Reactivity: How Public Measures Recreate Social Worlds,” *American Journal of Sociology*, vol. 113, no. 1, pp. 1–40, 2007, doi: [10.1086/517897](https://doi.org/10.1086/517897).
- [10] E. Strubell, A. Ganesh, and A. McCallum, “Energy and Policy Considerations for Deep Learning in NLP,” *arXiv preprint arXiv:1906.02243*, 2019.
- [11] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” *arXiv preprint arXiv:1907.10597*, 2019.
- [12] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, “Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning,” *arXiv preprint arXiv:2002.05651*, 2020.
- [13] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, “Quantifying the Carbon Emissions of Machine Learning,” *arXiv preprint arXiv:1910.09700*, 2019.
- [14] D. Patterson *et al.*, “Carbon Emissions and Large Neural Network Training,” *arXiv preprint arXiv:2104.10350*, 2021.
- [15] J. Dodge *et al.*, “Measuring the Carbon Intensity of AI in Cloud Instances,” *arXiv preprint arXiv:2206.05229*, 2022.
- [16] N. C. Thompson, K. Greenewald, K. Lee, and G. F. Manso, “The Computational Limits of Deep Learning,” *arXiv preprint arXiv:2007.05558*, 2020.
- [17] M. Dehghani, A. Arnab, L. Beyer, A. Vaswani, and Y. Tay, “The Efficiency Misnomer,” *arXiv preprint arXiv:2110.12894*, 2021.
- [18] J. Dodge, S. Gururangan, D. Card, R. Schwartz, and N. A. Smith, “Show Your Work: Improved Reporting of Experimental Results,” *arXiv preprint arXiv:1909.03004*, 2019.
- [19] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?,” *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT ’21)*, pp. 610–623, 2021, doi: [10.1145/3442188.3445922](https://doi.org/10.1145/3442188.3445922).